🏋️ GimUp – Smart Workout & Muscle Analysis App

> **Measure your training, not just your check-ins.**

A Flutter-based fitness application that goes beyond simple workout logging — GimUp helps you understand the **quality** and **muscle distribution** of every training session through training density analysis.

🚀 Overview

GimUp is a cross-platform gym training companion built with Flutter. It lets you build workout days, log sets and reps, and instantly see how much load each muscle group actually received — expressed as a concrete **training density** value (kg).

Instead of asking "did I train today?", GimUp helps you ask the better question: **"how well did I train, and which muscles did I really hit?"**

The app calculates density per exercise and aggregates it across every muscle involved, giving you a per-muscle breakdown that makes unbalanced training immediately visible.

🧠 The Idea Behind It

Most fitness apps let you check off workouts. Few of them tell you whether those workouts were any good.

GimUp is built around the concept of **Trainingsdichte** (training density): the total mechanical load delivered per muscle group, derived from sets × reps × weight. By weighting each exercise's muscle engagement percentages (e.g., Bench Press: 50% upper chest, 20% front shoulder, 15% tricep, 15% lower chest), the app distributes density across muscles realistically — not just by exercise name.

The result is a training tool that helps you:

- Spot when you are overloading one muscle group
- Notice when a muscle is consistently undertrained
- Compare session quality day to day with a single number
- Make deliberate decisions about your next workout based on data

---

✨ Features

🏗️ Workout Planning

- Build full training days from **46 predefined exercises** across 8 muscle-group categories
- Browse exercises organised by: **Chest, Back, Shoulders, Biceps, Triceps, Forearms, Legs, Core**
- Log multiple sets per exercise, each with individual reps and weight

📊 Training Density Analysis

- Per-exercise density calculated as `sets × reps × (weight + bodyWeight)`

- Per-muscle density derived from each exercise's weighted muscle-engagement percentages
- Day-level density table sorted by muscle impact
- Best training day tracked across your full history

💾 Saved Workout Templates

- Save any training day as a named template
- Reload a saved template with one tap to repeat a favourite session
- Delete templates you no longer need

🛠️ Custom Exercise Builder

- Create your own exercises and assign targeted muscles with custom engagement percentages
- Supports dumbbell mode (automatic ×2 density multiplier) and bodyweight percentage input
- Custom exercises persist locally and appear alongside standard ones

🗓️ Calendar & History

- Full calendar view (powered by `table_calendar`) showing every training date
- Tap any date to see the exercise list, set counts, and a complete density breakdown for that day
- Add new training days directly from the calendar

📈 Personal Statistics

- First and last training dates
- Total training days and total exercises performed
- Daily training average
- Best training day (by density)

🖼️ Exercise Images

- Every predefined exercise displays a reference image for quick identification

---

⚡ What Makes This Project Different

Typical Fitness App	GimUp
Logs what you did	Analyses how effective it was
Names muscle groups	Distributes load by percentage across sub-muscles
Counts reps	Calculates density in kg across 18 distinct muscle heads
Shows a history	Identifies your best sessions and reveals muscle imbalances

GimUp tracks **18 individual muscle segments**: upper/middle/lower chest, upper/middle/lower back, lats, front/side/rear shoulder, bicep, tricep, glutes, hamstrings, quads, calves, core, and forearms. This level of granularity is what allows meaningful training quality feedback rather than generic summaries.

🛠️ Tech Stack

Technology	Role
Flutter 3 / Dart 3	Cross-platform UI framework targeting Android, iOS, Web, Windows, Linux and macOS from a single codebase
flutter_riverpod 2	Reactive state management using `StateNotifier` — keeps UI in sync with app state without boilerplate
Hive + hive_flutter	Lightweight, fast NoSQL key-value store for offline-first local persistence of training days, saved templates, and custom exercises
table_calendar	Customisable calendar widget used to browse and navigate training history
Material Design	Standard Flutter UI components for a clean, familiar interface

All data is stored locally on the device — no account or internet connection required to use the app.

📱 Screenshots

> *Screenshots coming soon — run the app locally to see it in action.*

Home & Statistics	Calendar View	Training Density Table
home	calendar	density

📦 Installation

Prerequisites

- Flutter SDK ≥ 3.3.4
- Dart ≥ 3.3.4
- A connected device or emulator

Steps

```
# 1. Clone the repository
git clone https://github.com/housnizalam/gym_app.git
cd gym_app

# 2. Install dependencies
flutter pub get

# 3. Run the app
flutter run
```

To target a specific platform:

```
flutter run -d android
flutter run -d windows
flutter run -d chrome
```

📥 Download

Pre-built binaries:

- 📱 **Android APK** — *Download link coming soon*
- 🪟 **Windows installer** — *Download link coming soon*
- 🌐 **Web app** — *Hosted version coming soon*

🧩 Project Structure

```
lib/
├── main.dart                    # App entry point, Hive initialisation
├── models/
│   ├── appstate.dart            # Central app state + computed statistics
│   ├── day.dart                 # Training day model + density aggregation
│   ├── muscle.dart              # 18 individual muscle segment definitions
│   ├── sets.dart                # Single set (reps + weight)
│   ├── training.dart            # Core exercise model + density calculations
│   ├── training_images.dart     # Asset path constants for exercise images
│   ├── saved_trainingsday.dart # Named workout template model
│   ├── user.dart                # User profile model
│   └── trainings/               # 46 predefined exercises (one file per muscle
group)
│       ├── chest_trainings.dart
│       ├── back_trainings.dart
│       ├── shoulder_traininga.dart
│       ├── biceps_training.dart
│       ├── triceps_training.dart
│       ├── forarm.training.dart
│       ├── leg_trainings.dart
│       ├── core_trainings.dart
│       └── created_trining.dart  # Custom exercise model
├── riverpod/                    # State management providers
│   ├── appstate_provider.dart   # Main StateNotifier + CRUD operations
│   ├── calendar_providers.dart # Calendar selection state
│   ├── temp_day_provider.dart   # In-progress workout session state
│   ├── save_training_as.dart
│   ├── create_new_training.dart
│   └── body_weight.dart
├── storage/
│   ├── hive_main.dart           # HiveService: open boxes, load/save all data
│   └── json.converter.dart      # Manual JSON serialisation/deserialisation
├── gui/
│   ├── home_widgets/
```

```
│   │   ├── home.dart           # Main screen with statistics and navigation
│   │   ├── calendar_widget/    # Calendar screen + day detail view
│   │   ├── training_widget/    # Workout session flow (start/select/create)
│   │   └── statistic_tables/   # User statistics table widget
│   └── user_widgets/           # Sign-in and sign-up screens
└── globals/
    ├── widgets/                # Reusable UI components (tables, containers,
scaffold)
    └── extensionen/            # Dart extensions (DateTime, List, Map, Size,
Navigator)
```

🎯 Future Improvements

- **Progress charts** — visualise density trends over time with line/bar graphs
- **Muscle balance radar** — a visual map showing how evenly muscles are trained across a week
- **Cloud sync** — optional backup and restore via a remote backend
- **Rest timer** — integrated countdown between sets
- **Bodyweight tracking** — log and chart bodyweight over time alongside training density
- **Personalised recommendations** — suggest exercises based on undertrained muscle groups
- **Exercise search and filtering** — quickly find exercises by name or muscle
- **Dark / light theme toggle**

👨‍💻 About the Developer

GimUp was built by **Zalam Housni**, a Flutter developer focused on creating practical, well-structured mobile applications. This project reflects a deliberate approach to clean architecture — separating data models, state management, storage, and UI into clear layers — and a genuine interest in solving real-world problems rather than building demo apps.

The training density engine at the core of GimUp is an example of translating domain knowledge (exercise science) into a concrete data model that makes the app genuinely useful, not just functional.

Built with Flutter · Local-first · No account required